

Design of a New Stream Cipher—LEX

Alex Biryukov

University of Luxembourg, FSTC,
6, rue Richard Coudenhove-Kalergi,
L-1359 Luxembourg-Kirchberg Luxembourg

Abstract. In this paper we define a notion of leak extraction from a block cipher. We demonstrate this new concept on an example of AES. A result is LEX: a simple AES-based stream cipher which is at least 2.5 times faster than AES both in software and in hardware.

1 Introduction

In this paper we suggest a simple notion of a *leak extraction* from a block cipher. The idea is to extract parts of the internal state at certain rounds and give them as the output key stream (possibly after passing an additional filter function). This idea applies to any block cipher but a careful study by cryptanalyst is required in each particular case in order to decide which parts of the internal state may be given as output and at what frequency. This mainly depends on the strength of the cipher's round function and on the strength of the cipher's key-schedule. For example, ciphers with good diffusion might allow to output larger parts of the internal state at each round than ciphers with weak diffusion.

In this paper we describe our idea on an example of 128/192/256 bit key AES. Similar approach may be applied to the other block-ciphers, for example to Serpent. Interesting lessons learnt from LEX so far are that: LEX setup and resynchronization which are just a single AES key-setup and a single AES encryption are much faster than for most of the other stream ciphers (see performance evaluation of eSTREAM candidates [8]). This is due to the fact that many stream ciphers aimed at fast encryption speed have a huge state which takes very long time to initialize. Also, the state of the stream ciphers has to be at least double of the keysize in order to avoid tradeoff attacks, but on the other hand it does not have to be more than that. Moreover unlike in a typical stream cipher, where the whole state changes with time, in LEX as much as half of the state does not need to be changed or may evolve only very slowly.

2 Description of LEX

In this section we describe a 128-bit key stream cipher LEX (which stands for Leak EXtraction, and is pronounced “leks”). In what follows we assume that the reader is familiar with the Advanced Encryption Standard Algorithm (AES) [7]. The LEX design is very simple and is using AES in a natural way: at each

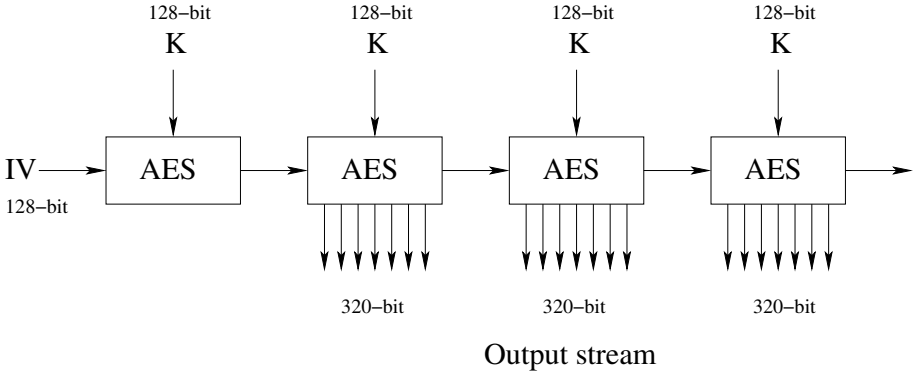


Fig. 1. Initialization and stream generation

AES round we output certain four bytes from the intermediate state. The AES with all three different key lengths (128, 192, 256) can be used. The difference with AES is that the attacker never sees the full 128-bit ciphertext but only portions of the intermediate state. Similar principle can be applied to any other block-cipher.

In Fig. 1 we show how the cipher is initialized and chained¹. First a standard AES key-schedule for some secret 128-bit key K is performed. Then a given 128-bit IV is encrypted by a single AES invocation: $S = AES_K(IV)$. The 128-bit result S together with the secret key K constitute a 256-bit secret state of the stream cipher.² S is changed by a round function of AES every round and K is kept unchanged (or in a more secure variant is changing every 500 AES encryptions).

The most crucial part of this design is the exact location of the four bytes of the internal state that are given as output as well as the frequency of outputs (every round, every second round, etc.). So far we suggest to use the bytes $b_{0,0}, b_{2,0}, b_{0,2}, b_{2,2}$ at every odd round and the bytes $b_{0,1}, b_{2,1}, b_{0,3}, b_{2,3}$ at every even round. We note that the order of bytes is not relevant for the security but is relevant for the fast software implementation. The order of bytes as given above allows to extract a 32-bit value from two 32-bit row variables t_0, t_2 in just four operations (that can be pipelined):

$$out32 = ((t_0 \& 0xFF00FF) \ll 8) \oplus (t_2 \& 0xFF00FF),$$

while each round of AES uses about 40 operations. Here t_i is a row of four bytes: $t_i = (b_{i,0}, b_{i,1}, b_{i,2}, b_{i,3})$. So far we do not propose to use any filter function and output the bytes as they are. The choice of the output byte locations (see also Fig. 2) is motivated by the following: both sets constitute an invariant subset of

¹ There is a small caveat: we use full AES to encrypt the IV, but we use AES with slightly modified last round for the stream generation, as will be explained further in this section.

² In fact the K part is expanded by the key-schedule into ten 128-bit subkeys.